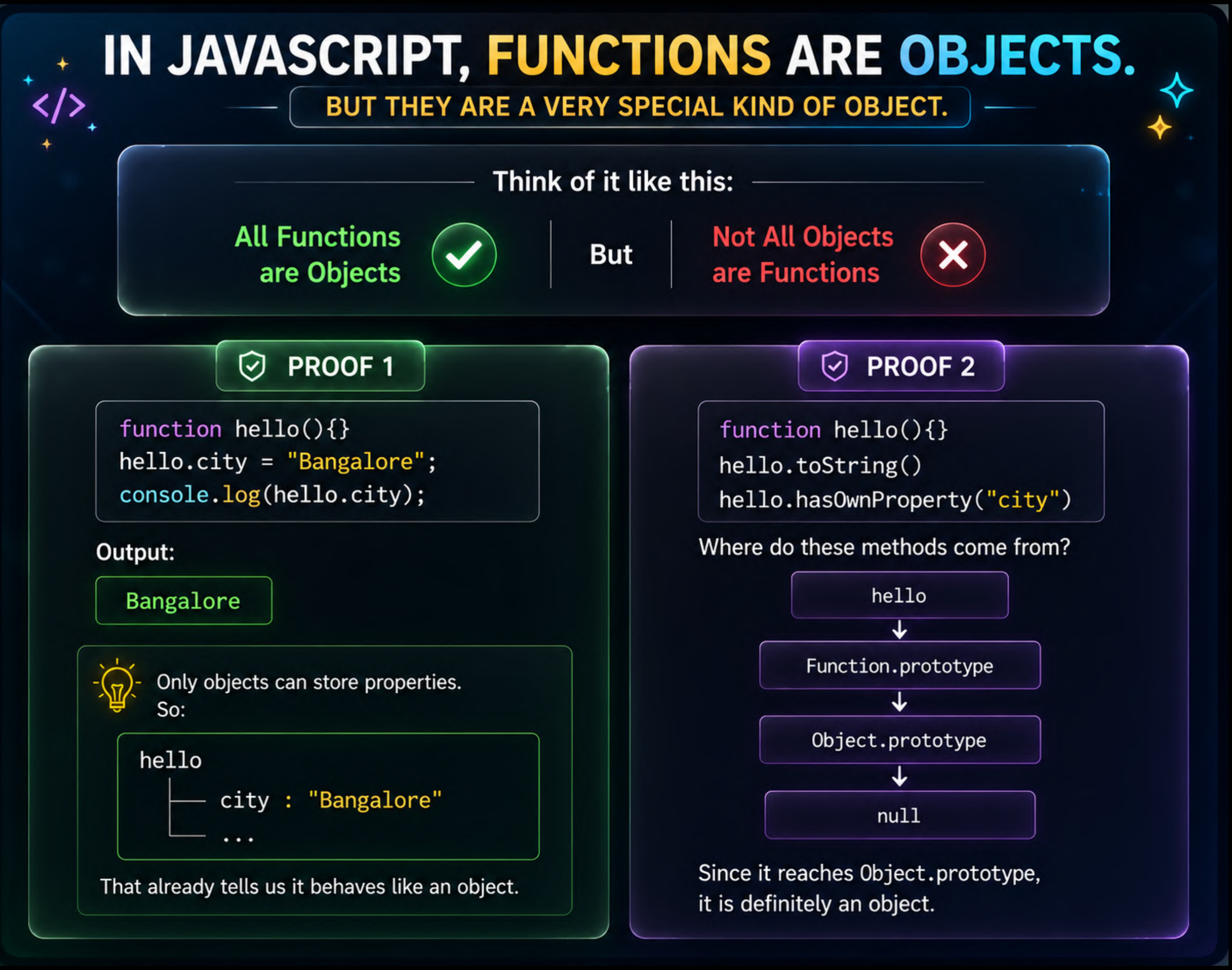
Hitesh Rathore (HR) Notes

https://www.linkedin.com/in/hitesh-rathore-835279305/



IN JAVASCRIPT, FUNCTIONS ARE OBJECTS.

BUT THEY ARE A VERY SPECIAL KIND OF OBJECT.

Think of it like this:

All Functions are Objects ✓ | But | Not All Objects are Functions ✗

PROOF 1

```
function hello(){}
hello.city = "Bangalore";
console.log(hello.city);
```

Output:

```
Bangalore
```

Only objects can store properties.
So:

```
hello
 ├── city : "Bangalore"
 └── ...
```

That already tells us it behaves like an object.

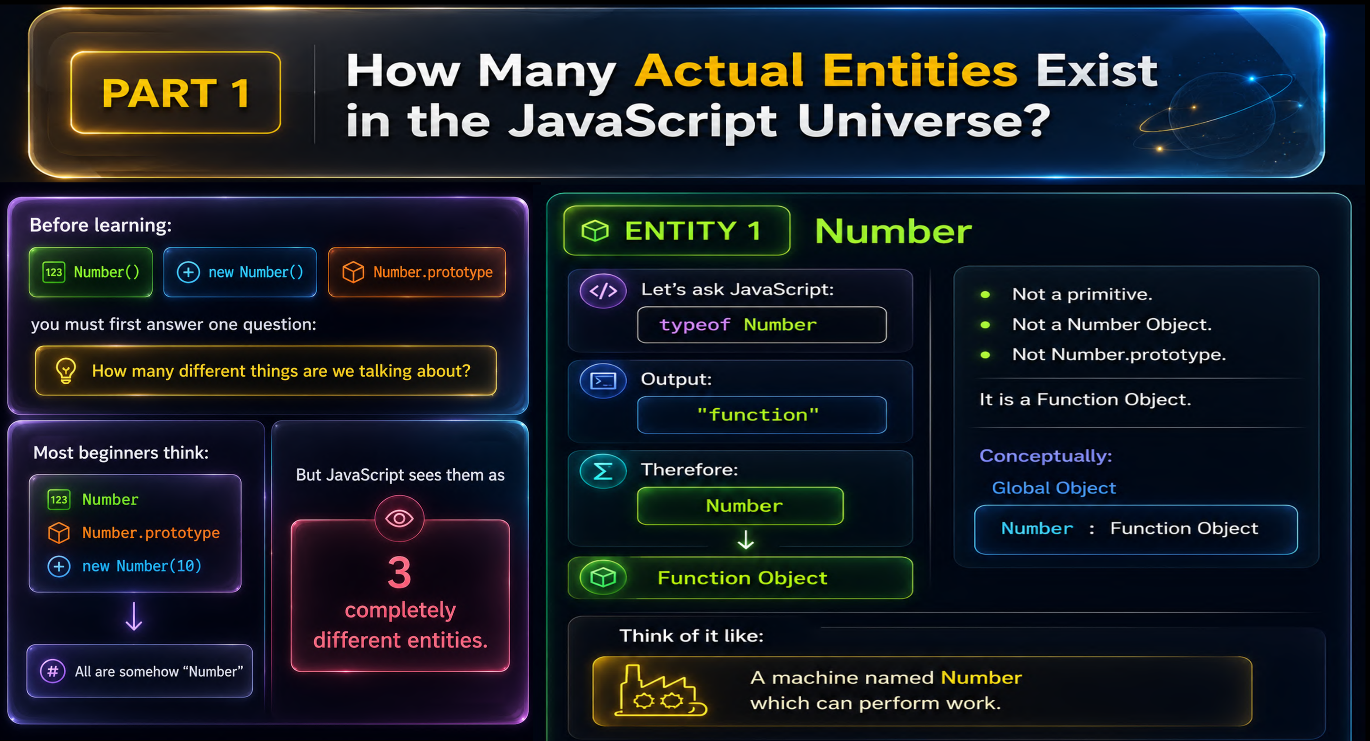
PROOF 2

```
function hello(){}
hello.toString()
hello.hasOwnProperty("city")
```

Where do these methods come from?

```
hello
  ↓
Function.prototype
  ↓
Object.prototype
  ↓
null
```

Since it reaches `Object.prototype`, it is definitely an object.



PART 1

How Many Actual Entities Exist in the JavaScript Universe?

Before learning:

`Number()` `new Number()` `Number.prototype`

you must first answer one question:

How many different things are we talking about?

Most beginners think:

- `Number`
- `Number.prototype`
- `new Number(10)`

↓

All are somehow "Number"

But JavaScript sees them as

3
completely different entities.

ENTITY 1 Number

Let's ask JavaScript:

```
typeof Number
```

Output:

```
"function"
```

Therefore:

Number
↓
Function Object

- Not a primitive.
- Not a Number Object.
- Not Number.prototype.

It is a Function Object.

Conceptually:

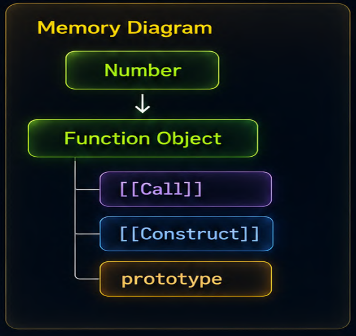
Global Object

Number : Function Object

Think of it like:

A machine named Number
which can perform work.

Memory Diagram



For now don't worry about:

- `[[Call]]`
- `[[Construct]]`
- `prototype`

We will study them later.
Just remember:

Number
↓
Function Object

ENTITY 2 Number.prototype

Now look at:

`Number.prototype`

Question:
Is this the same thing as Number?

No. Let's verify:

`Number === Number.prototype`

Output:

```
false
```

Therefore:

Number ≠ Number.prototype

What is Number.prototype?

Conceptually:

A normal object
that stores methods
for Number Objects.

Memory:

Number.prototype

- toFixed() 0.0
- toPrecision()
- valueOf() #
- toString() abc

Notice:

Number.prototype
↓
Object

NOT Function.

Let's verify:

`typeof Number.prototype`

Output:

```
"object"
```

So now we know:

Number
↓
Function Object

Number.prototype
↓
Ordinary Object

ENTITY 3 new Number(10)

Now:

```
let n = new Number(10);
```

Question:
What is created?

```
typeof n
```

Result:

```
"object"
```

Therefore:

new Number(10)
↓
Number Object

Conceptual Memory

n
↓
Object #1

[[NumberData]] : 10
[[Prototype]]

This object stores:
Wrapped Primitive Number
inside:

[[NumberData]]

VERY IMPORTANT COMPARISON

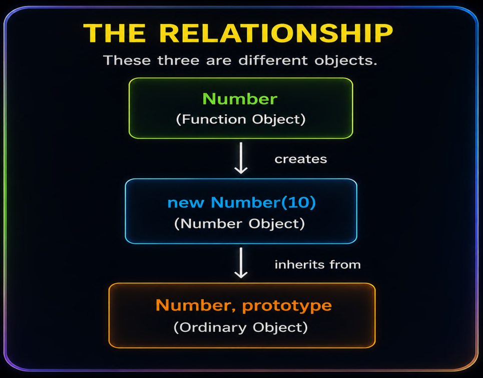
ENTITY #1	ENTITY #2	ENTITY #3
Number	Number.prototype	new Number(10)
Type: Function Object	Type: Ordinary Object	Type: Number Wrapper Object
Check: typeof Number	Check: typeof Number.prototype	Check: typeof new Number(10)
Output: "function"	Output: "object"	Output: "object"

MEMORY VIEW

ENTITY #1	ENTITY #2	ENTITY #3
Number ↓ Function Object	Number.prototype ↓ Ordinary Object	new Number(10) ↓ Number Object
[[Call]]	toFixed()	[[NumberData]] : 10
[[Construct]]	toPrecision()	[[Prototype]]
prototype	valueOf()	
	toString()	

THE RELATIONSHIP

These three are different objects.



MENTAL MODEL

Think of:

Number
as a Factory.

Think of:

new Number(10)
as a Product
made by the factory.

Think of:

Number.prototype
as a Shared Manual
attached to every Product.

PART 2

Why Is Number a Function?

Before learning JavaScript's answer, let's think from first principles.

THE REAL QUESTION

Suppose JavaScript did not have Number.

? **Question:**

What problems would JavaScript face?

Let's discover them.

</> PROBLEM 1

① Suppose you receive:

`"123"`

String

② Notice:

`"123"`

String

NOT Number

③ But sometimes we need:

`123`

Number

④ for calculations.

Example:

```
"123" + 1
```

Output: "1231"

because string concatenation happens.

⑤ But we may want:

`124`

instead.

⑥ So JavaScript needs something that can do:

String

↓

Number

conversion.

⑦ **Question:** Who should perform this conversion?

JavaScript creates: Number for this job.

Example:

```
Number("123")
```

Output: 123

⑧ **So first responsibility:**

Number

↓

Convert Values To Numbers

PROBLEM 2

JavaScript also supports wrapper objects.

Example:

```
new Number(10)
```

Question: Who should create this object?

JavaScript needs something capable of:

10

↓

Number Object

creation.

Again JavaScript chooses: Number for this job.

Example:

```
new Number(10)
```

Conceptually:

10

↓

Create Object

↓

Store 10 Inside [[NumberData]]

↓

Return Object

So second responsibility:

Number

↓

Create Number Objects

NOW THINK LIKE A LANGUAGE DESIGNER

Imagine you are designing JavaScript.

You need one thing that can:

Convert Values
AND
Create Number Objects

Question: What kind of thing should it be?

JavaScript's answer:

Function

because functions perform work.

Conceptually:

Number

↓

Machine

↓

Performs Jobs

WHY NOT AN ORDINARY OBJECT?

Imagine:

Number

↓

Ordinary Object

Then:

`Number("123")` would make no sense.

Because:

Objects ✗ Cannot Be Called

Functions ✓ Can Be Called

Example:

```
let obj = {};
obj();
```

Output: TypeError

because objects are not callable.

But:

```
function hello(){}
hello();
```

works.

Therefore:

If JavaScript wants: `Number(...)` to work,

Number must be callable.

Callable things are: Functions

FIRST IMPORTANT DISCOVERY

When you write:

`Number("123")`

JavaScript is literally calling a function.

Conceptually:

Call Number Function
↓
Convert Value
↓
Return Primitive Number

Visual:

"123"
↓
Number Function
↓
123

SECOND IMPORTANT DISCOVERY

When you write:

`new Number(123)`

JavaScript is still using:

The Same Function

Not another object.
Not another class.
Same function.

Visual:

Number
↓
Function Object
↓
Used in two ways:

Way 1: `Number("123")`
Way 2: `new Number("123")`

THE BIG SURPRISE

Most beginners imagine:

`Number()` ↓ One Thing ≠ `new Number()` ↓ Different Thing

Wrong.
Both use the same function.

Conceptually:

Number
↓
Function Object

Used differently.

RELATION WITH PART 1

In Part 1 we discovered:

Number
↓
Function Object

Now we know WHY.

Because JavaScript needed something capable of:

1. Converting values
2. Creating Number Objects

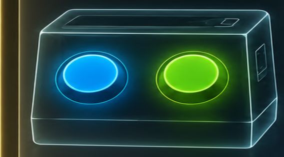
and functions are designed to perform work.

REAL WORLD ANALOGY

Imagine a machine.

Number Machine

It has two buttons.



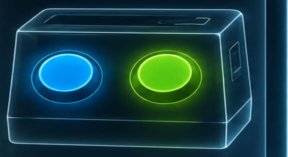
BUTTON 1
Convert

Input:
"123"
Output:
123
Equivalent to:
`Number("123")`

BUTTON 2
Create Object

Input:
123
Output:
Number Object
Equivalent to:
`new Number(123)`

Same machine.
Different mode.



PART 3

What Exactly Is a Function Object?

Until now we know:

`typeof Number`
↓
"function"

Number
↓
Function Object

But this creates a new question:

What is a Function Object?

Why doesn't typeof Number return "object"?

Let's start from absolute first principles.

1 WHAT IS AN OBJECT?

You already know:

```
let user = {
    name: "Hitesh",
    age: 20
};
```

Conceptually:

Object

name : ...
age : ...

An object is simply: **A container of data**

Example:

```
let car = { brand: "BMW" };
```

Conceptually:

Object

brand : BMW

Nothing surprising.

2 WHAT IS A FUNCTION?

Example:

```
function hello() {
    console.log("Hi");
}
```

Most beginners imagine:

Function
↓
Special thing
↓
Completely different from objects

But JavaScript does NOT think this way.

JavaScript thinks:

Function
↓
Special Object

This is one of the most important ideas in JavaScript.

Every Function is an Object

BUT

not every Object is a Function

Think:

Dog
↓
Animal

← But →

Animal
↓
Not always Dog

Function
↓
Object

← But →

Object
↓
Not always Function

3 PROOF THAT FUNCTIONS ARE OBJECTS

Can we store properties inside a function?

```
function hello(){
   // function body
}
hello.city = "Delhi";
console.log(hello.city);
```

Let's test.

Output:

```
Delhi
```

Question: Why does this work?

Because:

hello
↓
Object

Objects can store properties.

Therefore functions can store properties.

Conceptually:

hello

city : "Delhi"

Normal objects can do this. | Functions can do this.

Therefore functions are objects.

4 BUT FUNCTIONS HAVE A SUPERPOWER

Consider:

```
let user = {};
```

Can we do:

```
user();
```

Output:

```
TypeError
```

Why?

Because: Object
↓
Not Callable

Now:

```
function hello(){}
```

Can we do:

```
hello();
```

Output:

```
Yes.
```

Question:

Why can a function be called, but an object cannot?

Because a **function object** contains something **extra.**

5 THE SECRET INGREDIENT

Conceptually:

Normal Object

Object

Properties

Function Object

Function Object

Properties

[[Call]]

Notice:

[[Call]]
↓
This is the special power.

WHAT IS [[Call]]?

Think of it as:
Ability To Execute

When JavaScript sees:

```
hello()
```

Conceptually:

hello
↓
Use [[Call]]
↓
Execute Function Body

Without [[Call]]:

Object
↓
Cannot Be Called

✗

With [[Call]]:

Function Object
↓
Can Be Called

✓

6 NOW LOOK AT Number

Remember:

`typeof Number`

Output:

`"function"`

Meaning:

Number
↓
Function Object

Conceptually:

Number

`[[Call]]`

Since it has: `[[Call]]`

JavaScript allows:

`Number("123")`

Visual:

"123"
↓
Number Function
↓
[[Call]]
↓
123

7 ANOTHER SURPRISE

Functions can have properties too.

Example:

```
function hello(){}
hello.age = 100;
```

Conceptually:

hello

age : 100

[[Call]]

Notice:

Data
+
Callable Ability

Both together.
This is why we say:

Function
=
Special Object

8 WHY typeof RETURNS "function"

You may ask:
If functions are objects,
why not:

`typeof Number`
↓
`"object"` ?

So:

`typeof {}`

returns: `"object"`

Because JavaScript gives callable objects a special category.

Conceptually:

Object
↓
Has [[Call]]
↓
Function

but: `typeof function(){}`

returns: `"function"`

because it has: [[Call]]

CONCEPTUAL FLOW

Suppose:

```
function hello() {
  console.log("Hi");
}

hello();
```

Engine sees:

`hello()`
↓
Looks up:

`hello`
↓
Finds Function Object

Function Body
[[Call]]

Checks:
Does it have [[Call]] ?
↓
YES
↓
Invoke [[Call]]
↓
[[Call]] starts execution
↓
Create Execution Context
↓
Run Function Body
↓
`console.log("Hi")`
↓
Output: Hi

THEN WHAT IS [[Call]] EXACTLY?

According to the ECMAScript Specification:

[[Call]]

is an Internal Method.

Think:

- Object Methods
- toString()
- valueOf()

are visible methods.

But:

[[Call]]

is a hidden engine method.

Only the JavaScript engine, can invoke it.

You cannot do:

`hello.[[Call]]()`

✗

because it is internal.

WHAT IS STILL MISSING?

One mystery remains:

If Number only has [[Call]]

then how does:

`new Number(123)`

work?

A normal callable function should only execute.

It should NOT create objects.

That leads us to the next hidden ability:

[[Construct]]

How One Number Function Performs Two Completely Different Jobs

([[Call]] vs [[Construct]])

PART 4

How Can One Number Function Do Two Completely Different Jobs?

At this point we know:

Number → Function Object → Has [[Call]] → Can Be Invoked

Example:

`Number("123")` works because Number is callable.

But now a bigger mystery appears.

THE MYSTERY

This works:

`Number("123")`

← →

and this also works:

`new Number("123")`

Question:

How can the SAME function do two completely different things?

First returns:	Second returns:
`123` (a primitive number)	`new Number(123)` (a Number Object)

- Different outputs.
- Different behavior.
- Same function.

How?

1 THINK LIKE THE JAVASCRIPT ENGINE

Suppose JavaScript sees:

`Number("123")`

Question:

How does the engine know what to do?

Now suppose it sees:

`new Number("123")`

Question:

How does the engine know that object creation is needed?

The answer is:

The engine notices whether the keyword "new" is present.

Without new:	With new:
Normal Function Call	Constructor Call

2 FUNCTION OBJECTS HAVE TWO HIDDEN ABILITIES

Earlier we learned:

Function Object
[[Call]]

That picture was incomplete.

Many functions actually have:

Function Object
[[Call]]
[[Construct]]

Question:
What is [[Call]]?

Answer:
Ability To Run Like A Function

Example:
`Number("123")`

Question:
What is [[Construct]]?

Answer:
Ability To Create Objects when used with new

Example:
`new Number("123")`

3 CALL MODE

Consider:

`Number("123")`

No new.

Engine sees:

Number
↓
Function Object
↓
Use [[Call]]

Conceptually:

Number Function
↓
[[Call]]
↓
Convert "123"
↓
Return Primitive Number
↓
123

Visual:

"123"
↓
Number
↓
[[Call]]
↓
123

Output:

123

Check:

`typeof Number("123")`

Output:

"number"

Because:

[[Call]]
↓
Returned Primitive Number

4 CONSTRUCTOR MODE

Now:

`new Number("123")`

Engine sees:

Keyword: new

Immediately the engine changes behavior.

Instead of:

[[Call]]

it uses:

[[Construct]]

Conceptually:

Number Function
↓
[[Construct]]
↓
Create New Object
↓
Store 123 in [[NumberData]]
↓
Return Object

Visual:

"123"
↓
Number
↓
[[Construct]]
↓
Create Object
↓
[[NumberData]] = 123
↓
Return Object

Output:

[Number: 123]

(conceptually a Number Object)

Check:

`typeof new Number("123")`

Output:

"object"

Because:

[[Construct]]

Returned Object

5 SAME FUNCTION DIFFERENT ENTRY DOOR

Think of a building.

Number Building

It has two doors.

NUMBER
BUILDING

Door #1
[[Call]]
Used when:
`Number(...)`

Door #2
[[Construct]]
Used when:
`new Number(...)`

Visual:

Number

[[Call]] ↓ Convert Value

[[Construct]] ↓ Create Object

6 WHERE DOES Number.prototype ENTER?

This is where prototype chains begin.

Consider:

`new Number(10)`

Engine uses:

`[[Construct]]`

Question:
After creating the object,
what should be its prototype?

JavaScript answers:

`Number.prototype`

Conceptually:

Number Function
↓
has property
↓
`Number.prototype`

When object creation happens:

Create Object
↓
`Object.[[Prototype]]`
↓
`Number.prototype`

Visual:

Number Function

[[Call]]
[[Construct]]
prototype
↓
`Number.prototype`

7 FULL CONSTRUCTOR PROCESS

When JavaScript sees:

`new Number(10)`

Conceptually:

Step 1	Find Number Function
Step 2	Use [[Construct]]
Step 3	Create Empty Object
Step 4	Store 10 In [[NumberData]]
Step 5	Set Object.[[Prototype]]
Step 6	Return Object

↓

`Number.prototype`

Final Memory:

Object #1

[[NumberData]] : 10

[[Prototype]]
↓
`Number.prototype`

FINAL MENTAL MODEL

Number
↓
Function Object
↓
[[Call]]
[[Construct]]
prototype

Without new

Use [[Call]]
↓
Return Primitive Number

With new

Use [[Construct]]
↓
Create Number Object
↓
Set Prototype
↓
Number.prototype
↓
Return Object

THEN WHY DON'T ALL FUNCTIONS HAVE [[Construct]]?

Excellent question.

Suppose:

```
const add = (a,b) => a+b;
```

Question:
Should this create objects?

Imagine:

```
new add();
```

What object should be created?

Addition Object?
Makes no sense.

The purpose of this function is:

Calculate

not

Construct Objects

JAVASCRIPT'S DECISION

Arrow Function

[[Call]]

No: [[Construct]]

Therefore:

`new add();`
↓
Engine checks:

Does add have [[Construct]] ?
↓
NO
↓
Throw TypeError

Function Declaration

```
function greet() {
   console.log("Hello");
}
```

```
greet();
new greet();
```

Here:

- `[[Call]]` ✓
- `[[Construct]]` ✓

Function Expression

Function created and assigned to a variable.

```
const greet = function() {
   console.log("Hello");
};
```

```
greet();
new greet();
```

Here:

- `[[Call]]` ✓
- `[[Construct]]` ✓

“Regular Function”

This is a broad term that includes both of the above.

Declaration

```
function add(a, b) {
  return a + b;
}
```

Expression

```
const add = function(a, b) {
  return a + b;
};
```

✓ Both are regular functions.

Here:

- `[[Call]]` ✓
- `[[Construct]]` ✓

FINAL EVOLUTION OF THE MENTAL MODEL

ORDINARY OBJECT

Object

Properties

- Cannot Execute ✗
- Cannot Construct ✗

CALLABLE FUNCTION

Function

Properties
[[Call]]

- Can Execute ✓
- Cannot Construct ✗

Example:

```
const add = (a,b) => a+b;
```

CONSTRUCTOR FUNCTION

Function

Properties
[[Call]]
[[Construct]]

- Can Execute ✓
- Can Construct ✓

Examples:

- Number
- Object
- Array
- Date
- function Person(){}